



PDF Download
3695868.pdf
04 April 2026
Total Citations: 7
Total Downloads:
1283

Latest updates: <https://dl.acm.org/doi/10.1145/3695868>

RESEARCH-ARTICLE

Building a Coding Assistant via the Retrieval-Augmented Language Model

XINZE LI, Northeastern University, Shenyang, Liaoning, China

HANBIN WANG, Northeastern University, Shenyang, Liaoning, China

ZHENGHAO LIU, Northeastern University, Shenyang, Liaoning, China

SHI YU, Tsinghua University, Beijing, China

SHUO WANG, Tsinghua University, Beijing, China

YUKUN YAN, Tsinghua University, Beijing, China

[View all](#)

Open Access Support provided by:

[Northeastern University](#)

[Chinese Academy of Sciences](#)

[Tsinghua University](#)

Published: 17 January 2025

Online AM: 16 September 2024

Accepted: 31 August 2024

Revised: 25 July 2024

Received: 08 January 2024

[Citation in BibTeX format](#)

Building a Coding Assistant via the Retrieval-Augmented Language Model

XINZE LI, HANBIN WANG, and ZHENGHAO LIU, Northeastern University, Shenyang, China

SHI YU, SHUO WANG, and YUKUN YAN, Tsinghua University, Beijing, China

YUKAI FU, Chinese Academy of Sciences, Shenyang, China

YU GU and GE YU, Northeastern University, Shenyang, China

Pretrained language models have shown strong effectiveness in code-related tasks, such as code retrieval, code generation, code summarization, and code completion tasks. In this article, we propose C_od_e assista_Nt viA retrieval-augme_Nted language model (CONAN), which aims to build a code assistant by mimicking the knowledge-seeking behaviors of humans during coding. Specifically, it consists of a code structure-aware retriever (CONAN-R) and a dual-view code representation-based retrieval-augmented generation model (CONAN-G). CONAN-R pretrains CodeT5 using Code-Documentation Alignment and Masked Entity Prediction tasks to make language models code structure-aware and learn effective representations for code snippets and documentation. Then CONAN-G designs a dual-view code representation mechanism for implementing a retrieval-augmented code generation model. CONAN-G regards the code documentation

Xinze Li and Hanbin Wang contributed equally to this research.

This article is an extension of reference on ACL 2023. The previous conference version focused only on learning the representation of structured data to improve the performance of code retrieval. However, most of the existing code retrieval systems are combined with code generation tasks such as code generation, code summarization, and code completion to build code retrieval augmented frameworks. The knowledge boundary problem of the language model can be alleviated by retrieving relevant code snippets and documentation from external knowledge bases. Therefore, based on the previous work, we have made the following improvements and extensions: (1) we extend our previous SANTA model into a code assistant (CONAN), which consists of a code structure-aware retriever and a dual-view code representation-based retrieval-augmented generation model. (2) The dual-view code representation-based retrieval-augmented generation model designs a dual-view code representation mechanism that helps language models better understand code semantics by regarding the code documentation descriptions as prompts. (3) The dual-view code representation-based retrieval-augmented generation model employs the Fusion in Decoder (FID) architecture, which breaks the limitation of the input length of the language model. (4) The code assistant (CONAN) performs well on several code-related tasks including code retrieval, code generation, code summarization, and code completion. (5) CONAN can be used as an assistant for the large language models to assist them in finishing various code tasks. All codes are available at <https://github.com/NEUIR/CONAN>.

This work is supported by the Natural Science Foundation of China under Grant (No. 92267201, No. 62206042, and No. U23B2019), the Joint Funds of Natural Science Foundation of Liaoning Province (No. 2023-MSBA-081), and the Fundamental Research Funds for the Central Universities under Grant (No. N2416012).

Authors' Contact Information: Xinze Li, Northeastern University, Shenyang, China; e-mail: lxzlxz0716@gmail.com; Hanbin Wang, Northeastern University, Shenyang, China; e-mail: wanghanbinpanda@gmail.com; Zhenghao Liu (corresponding author), Northeastern University, Shenyang, China; e-mail: liuzhenghao@mail.neu.edu.cn; Shi Yu, Tsinghua University, Beijing, China; e-mail: yushi17@foxmail.com; Shuo Wang, Tsinghua University, Beijing, China; e-mail: wangshuo.thu@gmail.com; Yukun Yan, Tsinghua University, Beijing, China; e-mail: yanyk13@mails.tsinghua.edu.cn; Yukai Fu, Chinese Academy of Sciences, Shenyang, China; e-mail: fuyukai@sia.cn; Yu Gu, Northeastern University, Shenyang, China; e-mail: guyu@mail.neu.edu.cn; Ge Yu, Northeastern University, Shenyang, China; e-mail: yuge@mail.neu.edu.cn. Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1558-2868/2025/1-ART39

<https://doi.org/10.1145/3695868>

descriptions as prompts, which help language models better understand the code semantics. Our experiments show that CONAN achieves convincing performance on different code generation tasks and significantly outperforms previous retrieval augmented code generation models. Our further analyses show that CONAN learns tailored representations for both code snippets and documentation by aligning code-documentation data pairs and capturing structural semantics by masking and predicting entities in the code data. Additionally, the retrieved code snippets and documentation provide necessary information from both program language and natural language to assist the code generation process. CONAN can also be used as an assistant for Large Language Models (LLMs), providing LLMs with external knowledge in shorter code document lengths to improve their effectiveness on various code tasks. It shows the ability of CONAN to extract necessary information and help filter out the noise from retrieved code documents.

CCS Concepts: • **Information systems** → **Information retrieval**;

Additional Key Words and Phrases: Code Assistant, Code Generation, Code Retrieval, Retrieval Augmented Language Model

ACM Reference format:

Xinze Li, Hanbin Wang, Zhenghao Liu, Shi Yu, Shuo Wang, Yukun Yan, Yukai Fu, Yu Gu, and Ge Yu. 2025. Building a Coding Assistant via the Retrieval-Augmented Language Model. *ACM Trans. Inf. Syst.* 43, 2, Article 39 (January 2025), 25 pages.

<https://doi.org/10.1145/3695868>

1 Introduction

In recent years, the code pertaining technologies [11, 16, 66] have shown promising effectiveness in code-related tasks [40], such as code generation [18, 40, 48], code summarization [48, 65, 66], and code completion [39, 76]. This convincing generation effectiveness allows code developers to understand, modify, and write code more efficiently, making it possible to build an effective code assistant.

As shown in Figure 1, even though the pretraining technique improves the effectiveness of language models on code-oriented tasks, the code generation and understanding ability of language models is limited by the knowledge boundary of input **Program Language (PL)** or **Natural Language (NL)** which can result in them generating incorrect or unsatisfactory code [23, 42]. Similarly, in real software development scenarios, many professional software engineers also encounter challenging tasks that are beyond their capabilities and knowledge. Whenever this happens, software engineers usually seek related information from the question-answering forums or the code repository, such as StackOverflow and GitHub, facilitating them understand and write code [5, 57]. These software engineers not only refer to the code documentation and solutions but also copy the code segments (a code repository usually contains 7–23% cloned parts [61]) to increase their development productivity and accelerate software development [4, 29, 55].

Inspired by the above scenario, many works have begun to mimic the retrieval and generation behaviors of software engineers during the coding process to build the retrieval augmented model [36, 39, 48, 59, 60, 78] to improve the performance of language models in the code-related tasks. They utilize different knowledge sources of codes to benefit the code-related tasks [34], e.g., using related code segments [39, 48], code documentations [78], or external entities [59] to improve the quality of generated codes. These models employ BM25 or DPR [25] to retrieve related code segments and incorporate external coding knowledge by directly concatenating external information [39, 48]. The code segments are usually long, making the work only model two code segments while completing the codes [39]. Nevertheless, the code retrieval process inevitably introduces additional noise. In this case, it is crucial to alleviate the noise of retrieved code segments in building the retrieval augmented code generation model.

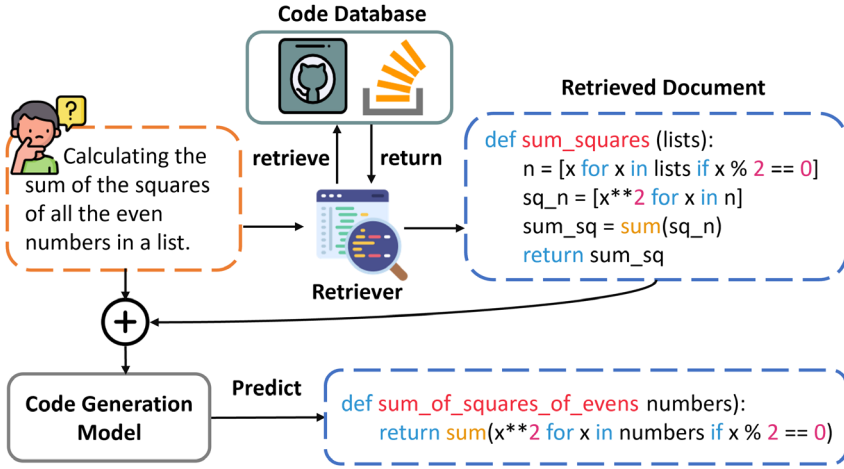


Fig. 1. The motivation of building a code assistant via the retrieval-augmented code generation model.

In this article, we propose **Code AssistaNt viA Retrieval-AugmeNted Language Model (CONAN)** to build a unified framework and serve code-related tasks, such as the code generation, code summarization, and code completion tasks. CONAN consists of a code structure-aware retriever (CONAN-R) and a dual-view code representation-based **Retrieval-Augmented Generation (RAG)** model (CONAN-G), which are designed to alleviate the noise of retrieved code segments. Specifically, CONAN-R reduces the noise of retrieved code knowledge by conducting more accurate retrieval results. Following our previous work [32], CONAN-R designs two pretraining tasks, **Code-Documentation Alignment (CDA)** and **Masked Entity Prediction (MEP)** to continuously train CodeT5 [66]. These tasks teach the language model to learn more effective representations for both code segments and documentation for retrieval. The CDA task contrastively trains **Pretrained Language Models (PLMs)** to align matched code-documentation pairs in the embedding space, which better represents codes by bridging the modality gap between PL and NL. The MEP task masks entities in codes and trains PLMs to fill in the masked parts, which helps to capture semantics from code. Then, to fully use the code knowledge provided by CONAN-R, CONAN-G follows previous work [60, 78] and employs the **Fusion-in-Decoder (FID)** architecture [22] to break the max length limitation of existing language models, making it possible to incorporate multiple retrieved code segments during generation. Besides, CONAN-G regards the code documentation as a gist, stimulates language models to capture more critical semantics from code structures using the code documentation, and alleviates the effect of noise from long code segments. Moreover, CONAN can be used as an assistant to provide **Large Language Models (LLMs)** with necessary code knowledge. Specifically, CONAN can retrieve relevant code documents from external knowledge databases and further summarize and denoise these retrieved code documents to shorter yet higher-quality documents, which in turn assists the LLMs in completing code-related tasks.

Our experiments show that both the retrieval module and generation module of CONAN achieve convincing performance in code-related tasks, such as code generation, code summarizing, and code completion, providing a promising way to build a code assistant. The effectiveness of CONAN mainly derives from more accurate code retrieval (CONAN-R) and the dual-view code representation-based RAG model (CONAN-G). On one hand, our further analyses show that CONAN-R achieves state-of-the-art on code retrieval tasks and shows strong zero-shot ability, which can supply more

informative code segments for CONAN-G. By aligning structured and unstructured data (CDA task), CONAN-R maps both codes and documentation in one universal embedding space and learns more tailored embeddings for code retrieval. The MEP task further guides CONAN-R to capture more crucial information for retrieval and better distinguish structured and unstructured data. On the other hand, by leveraging multiple code segments, CONAN-G outperforms baseline code generators and achieves consistent improvements in all code-related tasks. Notably, the code documentations show their effectiveness in guiding generation models to better capture key information from codes, further confirming that the multi-modal modeling method can generalize its advantages to the unimodal tasks [38].

2 Related Work

For building a code assistant, lots of work has focused on the code-related generation tasks [40], which include code generation [18, 40, 48], code summarization [48, 65, 66] and code completion [39, 76]. Recent work mainly focuses on pretraining language models to deal with code-related generation tasks, facilitating the code development [4, 29, 55]. To mimic the knowledge-seeking behavior of software engineers during coding, lots of work [36, 39, 48, 59, 60, 78] focuses on building the retrieval augmented model to improve the performance of language models in the code-related tasks. They utilize different information of codes to further improve the code-related tasks [34].

Code-Oriented Language Models. The code-oriented PLMs utilize code corpora for pretraining and design different training strategies to make the language model conduct a deeper understanding of code semantics, such as code syntax, semantics, and idiomatic constructs [1, 11, 75]. CodeBERT uses replaced token detection [8] and masked language modeling [9] to learn the lexical semantics of structured data [40]. DOBF [26] further considers the characteristics of code-related tasks and replaces class, function, and variable names with special tokens. CodeT5 [66] not only employs the span mask strategy [50] but also masks the identifiers in codes to teach T5 [50] to generate these identifiers, which helps better distinguish and comprehend the identifier information in code-related tasks. Additionally, some researchers leverage multi-modal data such as code, comment, and **Abstract Syntax Trees (AST)** to pretrain models, enhancing the model's understanding of code, NL, code structure, and other related information [15, 31]. Recently, LLMs, such as Llama [63] and ChatGPT [46], have demonstrated their ability in many code tasks, such as code understanding and code generation. To further improve the ability of LLMs on code tasks, many researchers focus on continuously pretraining LLMs on large amounts of code pretraining data to enable them to learn sufficient code knowledge. CodeQwen1.5 [62] is initialized from Qwen1.5 and trained on 3 trillion tokens of code data to make it with strong code generation capabilities. Code Llama [56] is pretrained based on Llama2 [63] with a total of 500 billion of generic and code data.

Code Retrieval. The code retrieval models [31, 32] usually employ the dense retrieval architecture for searching code segments [25, 33, 70, 73]. These models encode queries and codes using PLMs [9, 37, 50], map them in an embedding space for retrieval, and then conduct KNN search in the embedding space [24]. The query and code encoders are usually contrastively optimized to guarantee the retrieval effectiveness and the negatives are sampled from inbatch training documents, BM25 retrieved documents, and hard negatives [25, 69].

Learning more effective representations with PLMs is crucial for dense retrieval [13, 41]; thus several continuous training models are proposed. They usually employ mask language modeling to train PLMs on structured data and help to memorize the semantic knowledge using model parameters [11, 26, 66]. Nevertheless, the mask language modeling [9] may not sufficiently train PLMs to represent texts and show less effectiveness in text matching tasks [7, 12, 28, 28, 52]. The recent development of sentence representation learning methods has achieved convincing results [10, 71]. The work first constructs sentence pairs using back-translation [10], some easy

deformation operations [68], original sequence cropping [45], or adding dropout noise [14]. Then they contrastively train PLMs to learn sentence representations that can be used to distinguish the matched sentence pairs with similar semantics. Furthermore, some work also considers the characteristics of codes during pretraining. CodeRetriever [31] pretrains PLMs to learn more tailored representations for codes with the unimodal and bimodal contrastive losses, which encourages the model to push codes with similar functionality closer and align the matched code and text in the embedding space. We start from CodeT5 and propose code structure-aware pretraining [32] to pretrain language models structure-aware. Code structure-aware pretraining designs the CDA and MEP tasks for pretraining, which teach models to distinguish matched structured data for unstructured texts and ask language models to fill in the masked entities, respectively.

Retrieval-Augmented Code Generation. Code PLMs generate or complete codes based on input NL descriptions or code snippets [1, 39, 40]. However, existing code PLMs usually face the knowledge boundary problem of input [23, 42], which stimulates researchers to focus more on searching different knowledge for enhancing the code generation performance, e.g., using related code snippets [39, 48], code documentations [78], or external entities [59] to improve the quality of generated codes and learning necessary information from surrounding context to better understand a repository [60]. Even though the work leverages different kinds of knowledge for code generation, they incorporate external coding knowledge by directly concatenating external information [39, 48] or leverages the FID architecture [60, 78].

The external knowledge sources seem effective in enhancing the capabilities of generating more accurate code snippets, summaries, and completions. Specifically, REDCODER [48] retrieves relevant code snippets or summaries from a retrieval database using the DPR model [25] and then provides them as a supplement to improve the code generation and summarization performance. ReACC [39] focuses on the code completion task. It first utilizes the unfinished code as a query and then retrieves a similar code snippet that is completed using the lexical retrieval model. Then the unfinished code and completed code are concatenated and fed into the code generation model. SKCODER [30] is a sketch-based code generation approach, which extracts a code sketch from the retrieved similar code and further edits the sketch into the target code based on the input description. These existing methods still face challenges in fully using the retrieval information in the generation models. It is evident that the retrieval model returns lots of noise, which limits the effectiveness of code retrieval augmented models. Thus effectively searching and utilizing more related code context as auxiliary information is an ongoing research direction of code PLMs.

3 Methodology

In this section, we introduce the CONAN (Figure 2). We first introduce the preliminary of the retrieval augmented generation framework (Section 3.1). Then, we describe the retrieval module (CONAN-R) and generation module (CONAN-G) in Sections 3.2 and 3.3, respectively. CONAN-R pretrains PLMs to better capture the structure semantics of codes and conduct better code representations. CONAN-G utilizes the code documentation description as a gist to better understand code semantics. Finally, we utilize the CONAN model to generate code knowledge and aid LLMs for generating (Section 3.4).

3.1 Preliminary of the Retrieval Augmented Code Generation Framework

To deal with code generation, completion, and summarization tasks, CONAN regards the code function descriptions, unfinished codes, and code segments as queries q and then retrieves related code documents D as external knowledge to facilitate generating more accurate codes and summarizations. $d \in D$ and d consists of the code snippet d_{code} and the code documentation d_{doc} .

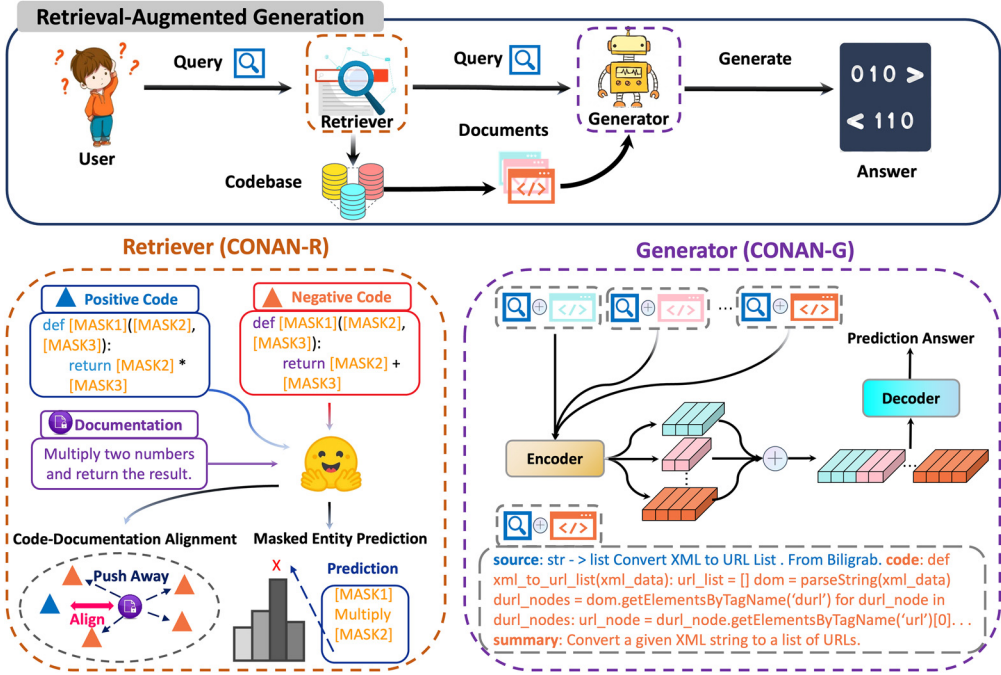


Fig. 2. The architecture of CONAN. CONAN consists of a code structure-aware retriever (CONAN-R) and a dual-view code representation mechanism (CONAN-G). We employ CDA and MEP methods for CONAN-R pretraining. CONAN-G is implemented with the FID architecture.

The retrieval augmented generation framework [19, 22, 27] includes a code retriever and a generation model, aiming to retrieve useful information for the generation model and utilize external knowledge to improve the generation accuracy. We utilize CodeT5 [65, 66] as backbone PLM and then implement the retrieval and generation modules.

Retrieval Module (CONAN-R). Existing retrieval augmented models usually leverage dense retrievers to conduct efficient search [19, 22, 27]. For the given query q , dense retrieval models aim to retrieve related code documents from the external code knowledge corpus. They encode the query q and document d and map them in an embedding space for retrieval. Following the previous work [32], we use CodeT5 to encode the query q and the document d as low-dimensional representations h^q and h^d , using the representation of the first token from the decoder:

$$h^q = \text{CodeT5}(q); h^d = \text{CodeT5}(d). \quad (1)$$

Then we conduct KNN search by calculating the similarity score $f(q, d)$ between the dense representations h^q and h^d of query q and document d :

$$f(q, d) = \text{sim}(h^q, h^d), \quad (2)$$

where sim is the dot product function to calculate the relevance between query q and document d . The top- N retrieved documents that are most similar to the query are denoted as $D = \{d^1, d^2, \dots, d^N\}$. Specifically, we utilize the code snippet d_{code} to represent the document d in the code generation task and use the code documentation d_{doc} to represent the document d in the code completion and code summarization tasks.

Generation Module (CONAN-G). To generate the code or NL sequence t , the generative module (CONAN-R) leverages the top- N retrieved documents $D = \{d^1, d^2, \dots, d^N\}$ from the retrieval model to facilitate the generation process.

To fully use the information from retrieved code documents D , we follow previous work [60, 78] and employ the FID architecture [22] to break the max length boundary of PLMs. The j th token t_j of the generated sequence t can be generated according to the probability $P(t_j|q, D, t_{1,\dots,j-1})$:

$$P(t_j|q, D, t_{1,\dots,j-1}) = \text{FID}(q, D, t_{1,\dots,j-1}). \quad (3)$$

Finally, the codes or summarization results can be generated.

3.2 CONAN-R with Structure-Aware Pretraining

To learn a tailored embedding space for code retrieval, CONAN-R finetunes the representations of query and document by minimizing the loss $\mathcal{L}_{\text{CONAN-R}}$:

$$\mathcal{L}_{\text{CONAN-R}} = -\log \frac{e^{f(q, d^+)}}{e^{f(q, d^+)} + \sum_{d^- \in D^-} e^{f(q, d^-)}}, \quad (4)$$

where d^+ is relevant to the given query q . D^- is the collection of irrelevant code documents, which are sampled from inbatch negatives [25]. Existing language models are usually pretrained on unstructured NLs with masked language modeling [9, 37]. Nevertheless, these models struggle to better understand the semantics represented by data structures, which limits the effectiveness of language models in representing code documents for retrieval [11, 66].

For CONAN, we follow the previous work [32] and continuously pretrain the CodeT5 to learn more tailored embedding space for codes using two structure-aware pretraining tasks: CDA and MEP. Through code structure-aware pretraining, PLMs further capture the structural semantics of the code and better learn the representation of code snippet, which can retrieve high-quality multi-view knowledge for the generator model.

CDA. The CDA task teaches language models to optimize the embedding space by aligning code snippet with documentation.

For each code snippet d_{code} , the document d usually contains the code documentation d_{doc} that has the same semantics as d_{code} . We can utilize the underlying semantic connections between these NL-based code documentation d_{doc} and code snippet d_{code} to perform alignment to train the language model to better represent code snippet.

Specifically, we can use CodeT5 to encode the code documentation d_{doc} and code snippet d_{code} as h_{code}^d and h_{doc}^d , respectively, calculate the similarity score $f(d_{\text{doc}}, d_{\text{code}})$ between d_{doc} and d_{code} , and then continuously train language models using the contrastive loss $\mathcal{L}_{\text{CDA}}$:

$$\mathcal{L}_{\text{CDA}} = -\log \frac{e^{f(d_{\text{doc}}, d_{\text{code}}^+)}}{e^{f(d_{\text{doc}}, d_{\text{code}}^+)} + \sum_{d_{\text{code}}^- \in D_{\text{code}}^-} e^{f(d_{\text{doc}}, d_{\text{code}}^-)}}, \quad (5)$$

where d_{code}^+ is relevant code snippet to the given d_{doc} . D_{code}^- consists of the irrelevant code snippet d_{code}^- sampled from inbatch negatives. The contrastive training method can bridge the semantic gap between code snippets and NL documentation and map them in one universal embedding space, benefiting learning representations of multi-modal text data [38].

MEP. The MEP guides the language models to better understand the semantics of code snippets by recovering masked entities. We mask entities for continuous training language models instead of using the random masking strategy in mask language modeling [9, 50].

As shown in previous work [58, 77], entity semantics show strong effectiveness in learning text data representations during retrieval. Thus, we first recognize mentioned entities that appeared in

the document $X_d = \{x_1, \text{ent}_1, x_2, \text{ent}_2, \dots, \text{ent}_n\}$ and mask them as the input for T5 encoder module:

$$X_d^{\text{mask}} = \{x_1, \text{mask}_1, x_2, \text{mask}_2, \dots, x_n\}, \quad (6)$$

where mask_i is a special token to denote the i th masked span. We replace the same entity with the same special token. Then we continuously train T5 to recover these masked entities using the following loss function:

$$\mathcal{L}_{\text{MEP}} = \sum_{j=1}^k -\log P(Y_d(t_j) | X_d^{\text{mask}}, Y_d(t_1, \dots, t_{j-1})), \quad (7)$$

where $Y_d(t_j)$ denotes the j th token in the sequence Y_d . And $Y_d = \{\text{mask}_1, \text{ent}_1, \dots, \text{mask}_n, \text{ent}_n\}$ denotes the ground truth sequence that contains masked entities. During training, we optimize the language model to fill up masked spans and better capture entity semantics by picking up the necessary information from contexts to recover the masked entities, understanding the structure semantics of code snippet [72].

3.3 CONAN-G with Dual-View Code Representation

As shown in Equation (3), we use the FID architecture to fully use the information of retrieved code documents D to benefit the code generation process. To optimize the parameters of CONAN-G, we train the model with the following loss function $\mathcal{L}_{\text{CONAN-G}}$:

$$\mathcal{L}_{\text{CONAN-G}} = \sum_{j=1}^m -\log P(t_j^* | q, D, t_1, \dots, t_{j-1}) = \sum_{j=1}^m -\log \text{FID}(q, D, t_1, \dots, t_{j-1}), \quad (8)$$

where t_j^* denotes the j th golden token of target sequence t and the sequence contains m tokens. The FID decoder module is inherited from CodeT5-Decoder. Then it uses the encoded representations $\text{Enc}(q, D)$ of query and retrieved documents and the embeddings $\{e_t^1, e_t^2, \dots, e_t^{j-1}\}$ of the tokens $\{t_1, t_2, \dots, t_{j-1}\}$ to calculate the generation probability of the next token t_j :

$$\text{FID}(q, D, t_1, \dots, t_{j-1}) = \text{CodeT5-Decoder}(\text{Enc}(q, D), \{e_t^1, e_t^2, \dots, e_t^{j-1}\}), \quad (9)$$

where the Enc function separately encodes the query q and code documents D using the CodeT5-Encoder:

$$\text{Enc}(q, D) = \text{CodeT5-Encoder}(d^1 \oplus q) \oplus \dots \oplus \text{CodeT5-Encoder}(d^N \oplus q), \quad (10)$$

where $\oplus$ is the concatenation operation.

For the document d^i , we can represent it using the code documentation d_{doc}^i and code segment d_{code}^i , which describe the function of the document in NL and PL, respectively. In CONAN, we propose a dual-view code representation method and simply concatenate the text sequences of the code documentation d_{doc}^i and code segment d_{code}^i to better represent the document:

$$\text{CodeT5-Encoder}(d^i \oplus q) = \text{CodeT5-Encoder}(d_{\text{doc}}^i \oplus d_{\text{code}}^i \oplus q). \quad (11)$$

The dual-view code representation method regards the code documentation d_{doc}^i as a kind of gist to help PLMs better understand code semantics. It thrives on the strong language understanding ability of PLMs and then utilizes functional instruction to capture crucial information from the code structure.

Table 1. Data Statistics of Pretraining Data

Task	Positive Pairs	Entities
Python	429,596	28.6%
PHP	514,127	17.8%
Go	317,824	17.1%
Java	454,433	24.4%
JavaScript	122,682	15.4%
Ruby	48,790	28.8%

“Entities” denotes the proportion of identified entities in the code data.

3.4 Assisting LLMs for Code-Related Tasks Using CONAN

Besides dealing with different code-related tasks, CONAN can also be used as a code assistant to aid LLMs for generating codes. In this case, CONAN aims to retrieve code segments and then extract necessary knowledge from retrieved code segments, enabling LLMs to access the external code knowledge and filter out the noise from retrieved contents.

Specifically, for a query q , CONAN first uses the query q to retrieve the relevant code documents D from the whole database $\tilde{D}$:

$$D = \text{CONAN-R}(q, \tilde{D}), \quad (12)$$

where CONAN-R is the retrieval module of CONAN. Then, we use CONAN-G to extract the code knowledge from these retrieved code documents D by generating a new code document d^* :

$$d^* = \text{CONAN-G}(q, D), \quad (13)$$

where d^* represents the summarization results, code snippet, and code segments for code summarization task, code completion task, and code generation task, respectively. Finally, we use d^* as the augmented knowledge for the code LLMs to assist LLMs to generate outputs y during solving different code-related tasks:

$$y = \text{LLM}(d^* \oplus q), \quad (14)$$

where we regard the generated code knowledge d^* as the context and concatenate it with the given query q to support the inference of LLMs.

4 Experimental Methodology

In this section, we describe the datasets, retrieval databases, evaluation metrics, baselines, and implementation details of our experiments.

4.1 Dataset

In this subsection, we introduce the datasets used in pretraining CONAN-R and code-related generation tasks.

Retrieval. Firstly, we present the pretraining data CodeSearchNet, which is used to pretrain our CONAN-R model. The data statistics are shown in Table 1.

During pretraining CONAN-R, we use the CodeSearchNet in experiments. As shown in Figure 3, we present an example to show how to construct the code-documentation pairs for pretraining. The code snippets have corresponding code documentations, which describe the purpose and function of these code snippets. As shown in Figure 3(a), the code documentation and its corresponding code snippet are regarded as a training pair. Then we regard the documentation as a query and

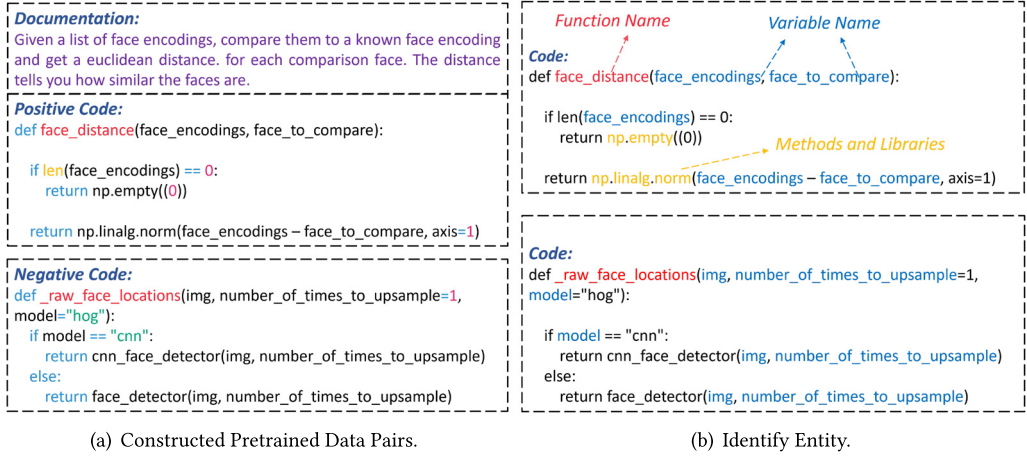


Fig. 3. Examples of the pretraining data for CONAN-R. All entities of different functions are annotated with different colors in Figure 3(b).

Table 2. Dataset Statistics for Code-Related Generation Tasks

Task	Dataset	Lang	Train	Dev	Test	Code	Doc
Code Generation	CgCSN	Python	251,820	13,914	14,918	99	14
		Java	164,923	5,183	10,955	97	12
	Concode	Java	100,000	2,000	2,000	27	72
	HumanEval	Python	-	-	164	66	156
	MBPP	Python	-	-	500	17	78
Code Summarization	CsCSN	Python	251,820	13,914	14,918	99	14
		Java	164,923	5,183	10,955	97	12
Code Completion	PY150	Python	68,589	3,825	10,000	497	7
	JavaCorpus	Java	11,774	4,993	3,000	639	11

|Code| and |Doc| represent the average lengths of code and documentation in the dataset for code generation and summarization tasks, respectively. In code completion, they respectively represent the average length of incomplete code and the length of code to be completed. All the code and documentation lengths are calculated before tokenization.

use inbatch negatives to optimize T5 for code retrieval pretraining. Additionally, as shown in Figure 3(b), we follow Wang et al. [66] and regard code identifiers such as variables, function names, external libraries, and methods as entities. Then we replace the same entities with the same special tokens and ask CONAN-R to generate these masked entities (Equation (7)). These special tokens come from the vocabulary of T5, such as {<extra_id_0>, <extra_id_1>, ..., <extra_id_99>}. BytesIO and tree_sitter¹ are utilized to identify entities in Python and other programming languages, respectively. The proportions of identified entities in pretraining data are shown in Table 1.

Code-Related Generation. We describe the datasets used to evaluate the generation effectiveness of CONAN-G, including code generation, code summarization, and code completion datasets. The data statistics are shown in Table 2.

Code Generation. The code generation task aims to generate the code snippets according to the given code documentation description. We first utilize the Concode [21] and CgCSN [48] datasets

¹<https://github.com/tree-sitter/tree-sitter>

Table 3. The Statistics of the Retrieval Database

Database	Lang	Task	Total	Paired	Unpaired
Code Snippets	Python	CgCSN-Python	1.2M	696K	507K
	Java	CgCSN-Java	1.6M	1.1M	0.5M
	Java ^a	Concode	104K	104K	-
Code Documentation	Python	CsCSN-Python and PY150	1.1M	267K	833K
	Java	CsCSN-Java and JavaCorpus	1.1M	197K	903K

“Paired” indicates that the code snippet/documentation in the retrieval database has corresponding code documentation/snippet. “Unpaired” indicates that the code snippet/documentation in the retrieval database does not have corresponding code documentation/snippet. “a” signifies that code candidates include not only Java code but also class environments.

to evaluate the code generation ability of different models. For the Concode [21] dataset, the input not only includes an NL description but also encompasses the class environment. This dataset is particularly challenging because the desired code can vary significantly based on the functionality provided by the class. Besides, we follow previous work [48] and use the CgCSN dataset in the code summarization task. CgCSN dataset is filtered from CodeSearchNet [20]. Additionally, we use HumanEval [6] and MBPP [3] to further test the ability of CONAN to assist the LLMs to generate codes. This dataset focuses more on evaluating the code generation ability of models whether they can pass the test cases.

Code Summarization. The code summarization task is a reversed task of code generation, which generates a code documentation description according to the code snippet. In the code summarization task, we use the CsCSN dataset in our experiments, which is filtered from CodeSearchNet [20] and includes two programming languages, Python and Java. Some examples that the code cannot be parsed into an AST have been removed. This preprocessing method ensures that the dataset keeps a high quality.

Code Completion. The code completion task targets completing unfinished codes. In this experiment, we employ the PY150 [51] and JavaCorpus [2] datasets to evaluate the generation performance. In our experiments, we specifically focus on line-level code completion, which auto-complete a line based on the provided incomplete code snippet.

Retrieval Databases. We follow Parvez et al. [48] to construct two retrieval databases, namely the (1) *code snippets retrieval database* and (2) *code documentation retrieval database*. In our experiments, we exclude the target code/documentation from the retrieval database to prevent information leakage from the generation dataset. The statistical information is shown in Table 3.

Code Snippet. The code snippet corpus is built based on CodeSearchNet [20]. After deduplication, the code retrieval database contains 1.2 million Python functions and 1.6 million Java functions. Approximately 40% of these functions are paired with corresponding NL descriptions. For Concode dataset, we merge its training and validation sets to create a code retrieval database, making all code snippets have corresponding NL descriptions.

Code Documentation. The code documentation corpus is built based on the combination of high-quality NL documentation from both CodeSearchNet and CCSD [36]. After deduplication, we retained 1.1 million code documentations, with approximately 20% of them containing corresponding Java and Python code snippets.

4.2 Evaluation Metrics

In this subsection, we describe the evaluation metrics to test the retrieval and generation performance of CONAN.

Table 4. Data Statistics of Code Retrieval Datasets

Dataset	Language	Train	Dev	Test
Adv	Python	251,820	9,604	19,210
CodeSearch	Python	251,820	13,914	14,918
	PHP	241,241	12,982	14,014
	Go	167,288	7,325	8,122
	Java	164,923	5,183	10,955
	JavaScript	58,025	3,885	3,291
	Ruby	24,927	1,400	1,261

Two datasets, Adv and CodeSearchNet, are used in our experiments.

To evaluate the retrieval performance of CONAN-R, we finetune CONAN-R and then evaluate its retrieval effectiveness using two code retrieval datasets, Adv [40] and CodeSearch, which are filtered out from CodeSearchNet dataset [20]. The data statistics of the finetuning data are shown in Table 4. CodeSearch consists of code retrieval tasks on six programming languages, including Ruby, JavaScript, Go, Python, Java, and PHP, which can evaluate the model's performance across a diverse set of programming languages. We use MRR@100 to evaluate the performance of the structure-aware code retriever CONAN-R, which is the same as the previous work [15, 31, 40].

To evaluate the generation performance of CONAN-G, we utilize different evaluation metrics for different tasks. We utilize the corpus level BLEU [47], CodeBLEU (CBLEU) [53], and Pass@ k as the evaluation metrics for code generation. We use the smoothed BLEU-4 [35] as the evaluation metric for code summarization. For code completion tasks, we adopt **Exact Match Accuracy (EM)** and **Edit Similarity (ES)** to evaluate line-level code completion.

4.3 Baselines

In this subsection, we describe the baselines used in our experiments. We evaluate CONAN against several state-of-the-art code-related generation models and code retrieval models. All baseline models are categorized into two groups: (1) retrieval models and (2) generation models.

Code Retrieval Models. We compare CONAN-R with three typical and task-specific code retrieval models to demonstrate its retrieval effectiveness, CodeBERT, CodeT5, and CodeRetriever [31]. CodeRetriever is the state-of-the-art code retrieval models, which continuously trains GraphCodeBERT [16] with unimodal and bimodal contrastive training losses.

Code Generation Models. The code generation models can be grouped into four categories, including retrieval models, PLMs, PLM w. RAG models, and LLMs.

Retrieval Models. Following prior work [48], we use the top-ranked code snippets/documentation from the retrieval results as the prediction results. We consider the term-based sparse retriever BM25 [54] and three dense retrievers, CodeBERT [11], GraphCodeBERT [16], and SCORE-R [48] as baseline models. CodeBERT inherits the BERT architecture and is trained on code corpus using both mask language modeling and replaced token detection. GraphCodeBERT is pretrained by modeling the dataflow graph of the source code. SCORE-R builds upon the DPR [25] model and uses CodeBERT and GraphCodeBERT as the code and summary encoders.

PLMs. The generative model produces the output based on the original input, without external information. CodeGPT and CodeGPT-adapted [40] are both decoder-only transformer models pretrained on Python and Java datasets from CodeSearchNet. The former is trained from scratch, while the latter is obtained by continuously training from GPT-2. PLBART [1] is a seq2seq model that is capable of performing a broad spectrum of program and language understanding and generation

tasks. CodeT5 [66] bases on the T5 architecture. It not only has excellent code understanding capabilities but also possesses strong code generation abilities. UniXcoder [15] is a unified cross-modal pretrained model that leverages multi-modal data (i.e., code comment and AST) to pretrain code representations.

PLM w. RAG. These models utilize different retrieval techniques to gather relevant information from a retrieval database, which is then used to guide and enhance the generative models. REDCODER [48] is a framework that retrieves relevant code snippets or documentation from a retrieval database and provides them as a supplement to code generation or summarization models. ReACC [39] is a retrieval-augmented code completion framework. It adopts a stage-wise approach that combines a source code retriever and an auto-regressive language model for programming language. ReACC-bm25, ReACC-dense, and ReACC-hybrid are three implementations of ReACC, each of which employs a different retriever.

LLMs. Moreover, we consider CONAN as an assistant to help LLMs solve different code tasks. Specifically, we use CONAN-R to retrieve relevant code snippets and documentation from external knowledge bases. And then we use CONAN-G to summarize and denoise the retrieved contents to obtain higher-quality knowledge, which is used to assist code LLMs. In our experiments, we use Deepseek-Coder-6.7b-Instruct (DSCoder-6.7b-Ins) [17] and CodeQwen1.5-7B-Chat (CQwen1.5-7B-Chat) [62] as code LLMs.

4.4 Implementation Details

In this subsection, we describe the experimental details of CONAN.

We initialize CONAN-R with CodeT5-base. During the structure-aware pretraining, we set the learning rate as $1e-4$ and the training epoch as 10. During finetuning, we train CONAN-R using inbatch negatives and hard negatives. For CodeSearch and Adv datasets, we set the learning rate as $2e-5$ and $1e-5$, respectively, and set batch size and epoch as 128 and 12. We use inbatch negatives plus one hard negative for finetuning and the hard negative is randomly sampled from the top-100 retrieved negative codes by the finetuned CONAN-R (Inbatch) model. For Concode and CsCSN, we set the learning rate as $2e-5$, while for CgCSN, we set the learning rate as $1e-5$. For all three datasets, we set batch size and epoch as 64 and 10. We use the Adam optimizer and set the warmup proportion as 0.1. All models are implemented with OpenMatch [74].

We initialize CONAN-G based on CodeT5-base with the FID architecture. CONAN-G utilizes a dual-view code representation method, which regards the code documentation description as a gist. However, some documents do not contain the code documentation. Thus, for these instances, we directly use the code snippets to represent the code documents. During training CONAN-G for all code-related generation tasks, we use the retrieved top-5 code snippets and documentation as external knowledge. On the Concode dataset, we set the learning rate as $1e-4$. For other datasets, we set the learning rate as $5e-5$. For all datasets, we set the batch size as 1, set max epoch as 1, use the AdamW optimizer, and configure the warmup steps as 1,000. All models are implemented with PyTorch and HuggingFace transformers [67]. When evaluating LLMs on HumanEval and MBPP, we set the temperature to 0.2 and the maximum generation length to 512 tokens.

5 Evaluation Result

In this section, we first explore the performance of CONAN on different code-related generation tasks and verify the denoising effect of CONAN. Then, we conduct ablation studies to show the effectiveness of different modules in CONAN. The effectiveness of structure-aware retriever pretraining and dual-view code representation-based RAG models are presented. Finally, case studies are shown.

Table 5. Evaluation Results of Code Generation and Code Summarization on Concode and CsCSN Datasets

Setting	Models	Code Generation Concode			Code Summarization CsCSN-P CsCSN-J	
		EM	BLEU	CBLEU	BLEU	BLEU
Retrieval Models	BM25	0	20.3	23.7	1.9	1.8
	CodeBERT [11]	0	27.7	41.4	11.6	12.1
	CodeT5 [66]	0	31.1	34.9	14.6	15.7
	SCODE-R [48]	0	32.6	36.5	15.0	15.9
	CONAN-R	0	33.5	37.5	15.6	16.2
PLMs	Seq2Seq [43]	3.1	21.3	26.4	15.9	15.1
	GPT-2 [49]	17.4	25.4	29.7	-	-
	CodeGPT-2 [40]	18.3	28.7	32.7	-	-
	CodeGPT-adapted [40]	20.1	32.8	36.0	-	-
	CodeBERT [11]	18.0	28.7	31.4	19.1	17.7
	GraphCodeBERT [16]	18.7	33.4	35.9	18.0	17.9
	PLBART [1]	18.6	36.7	38.5	19.3	18.5
	UniXcoder [15]	22.6	38.2	-	19.3	-
	CodeT5 (Ours) [66]	22.2	39.6	43.8	20.4	20.5
PLM w. RAG	BM25 + PLBART [48]	21.4	40.2	41.8	19.6	19.7
	REDCODER [48]	23.4	41.6	43.4	21.0	22.9
	REDCODER-EXT [48]	23.3	42.5	43.4	20.9	22.9
	CONAN	23.1	42.8	45.1	23.5	26.5
LLMs	DSCoder-6.7b-Ins	0	7.7	12.9	5.1	4.4
	DSCoder-6.7b-Ins + CONAN-R	14.1	12.8	38.2	18.8	19.8
	DSCoder-6.7b-Ins + CONAN	24.2	42.4	45.8	23.9	27.1
	CQwen1.5-7B-Chat	0	8.5	16.5	3.2	4.6
	CQwen1.5-7B-Chat + CONAN-R	18.0	30.5	38.5	4.8	5.2
	CQwen1.5-7B-Chat + CONAN	24.2	43.1	44.8	19.7	24.8

CsCSN-P and CsCSN-J represent the subsets of the CsCSN dataset to evaluate the code summarization effectiveness in Python and Java programming languages. The baseline results of PLMs setting are reported from PLBART [1] and REDCODER [48].

5.1 Overall Performance

In this subsection, we show the overall performance of CONAN on code-related generation tasks, including code generation, code summarization, and code completion.

As shown in Table 5, we first show the code generation and code summarization performance of CONAN on the Concode and CsCSN datasets. The code generation and code summarization tasks generate code snippets and code documentation descriptions according to the code documentation descriptions and code snippets, which aims to estimate the code understanding and generation ability. Overall, CONAN achieves the highest BLEU and CBLEU scores among all baseline models and also surpasses the state-of-the-art models REDCODER-EXT with an average of approximately 3.1% and 0.6% improvements on CsCSN and Concode datasets, respectively. It shows the effectiveness of our CONAN model.

For the baseline models, the models that are in Retrieval Models setting even outperform the CodeGPT2 model, demonstrating that the retrieved code snippets and documentation can help to answer the given question. It confirms the crucial roles of retrieved code snippets, which have many overlaps with the ground truth answers. Among all models in Retrieval Models setting, CONAN-R outperforms BM25, CodeBERT, GraphCodeBERT, CodeT5, and SCODE-R in terms of BLEU and

Table 6. Code Generation Results on the CgCSN Dataset

Setting	Model	Python			Java		
		EM	BLEU	CBLEU	EM	BLEU	CBLEU
Retrieval Models	BM25	0	6.6	13.5	0	4.9	16.0
	CodeBERT [11]	0	19.8	20.1	0	21.3	22.8
	CodeT5 [66]	0	23.1	23.3	0	26.0	27.1
	SCODE-R [48]	0	22.8	23.9	0	25.3	26.7
	CONAN-R	0	25.0	25.6	0	28.1	31.6
PLMs	CodeBERT [11]	0	4.1	10.4	0	8.4	14.5
	GraphCodeBERT [16]	0	4.0	10.6	0	7.9	14.5
	CodeGPT-adapted [40]	0	3.1	11.3	0	7.1	14.9
	PLBART [1]	0	4.9	12.0	0	10.1	15.0
	CodeT5 (Ours) [66]	0	6.3	14.8	0	12.2	17.8
PLM w. RAG	BM25 + PLBART [48]	0	7.0	13.9	0.1	11.4	15.5
	REDCODER [48]	8.9	22.7	28.9	9.0	26.9	31.2
	REDCODER-EXT [48]	9.6	24.4	30.2	10.2	29.0	33.2
	CONAN	14.6	32.9	37.3	17.2	37.7	45.4
LLMs	DSCoder-6.7b-Ins	0	4.2	10.2	0	6.4	13.3
	DSCoder-6.7b-Ins + CONAN-R	2.2	5.9	16.6	4.3	13.7	24.7
	DSCoder-6.7b-Ins + CONAN	20.5	33.2	37.3	21.4	38.1	45.5
	CQwen1.5-7B-Chat	0	5.7	9.8	0	9.8	20.5
	CQwen1.5-7B-Chat + CONAN-R	1.34	7.9	15.6	2.6	10.5	21.3
	CQwen1.5-7B-Chat + CONAN	19.4	32.9	37.1	22.5	38.1	46.2

The baseline results of PLMs' setting are reported from REDCODER [48].

CBLEU scores for both Python and Java programming languages of both code generation and summarization tasks. This indicates the effectiveness of CONAN-R in retrieving more relevant code snippets or documentation descriptions for the given queries. The reason for the EM value being 0 is that we filter out the ground truth answers from the retrieval results. We do this because, in real-world scenarios, the ground truth answers are rarely exactly matched with the retrieved code snippets. Thrived on external knowledge, the retrieval augmented model shows much better performance than the vanilla code/summarization generation models. CONAN achieves more than 2% improvements than our main baseline model CodeT5, which illustrates that CONAN can use the external code knowledge for generating. Besides, CONAN also outperforms all models in PLM w. RAG setting, showing the effectiveness of the code-aware pretraining method for CONAN-R and the dual-view code representation method for CONAN-G. When utilizing CONAN as an assistant for the code LLMs setting, CONAN-R can retrieve external knowledge to improve the performance of code LLMs on Concode and CsCSN datasets (DSCoder-6.7b-Ins + CONAN-R and CQwen1.5-7B-Chat + CONAN-R). In addition, when we use CONAN-G to summarize and denoise this retrieved external knowledge, the effectiveness of the code LLMs on Concode and CsCSN is further improved (DSCoder-6.7b-Ins + CONAN and CQwen1.5-7B-Chat + CONAN), achieving an approximately 10% improvement in EM. This indicates that CONAN can refine the retrieved knowledge to improve the performance of code LLMs by filtering out noise and irrelevant information.

Then, as shown in Table 6, we further evaluate the CONAN model on the CgCSN dataset. The average code length of this dataset is 98, which is much longer than the Concode dataset which is 27. CONAN achieves more significant improvements on the CgCSN dataset than the performance on the Concode dataset (approximately 7% improvements on CgCSN and 0.6% improvements on Concode). The improvements demonstrate that CONAN has the ability to better understand the

Table 7. Evaluation Results on the Code Completion Task

Setting	Model	PY150		JavaCorpus	
		EM	ES	EM	ES
PLMs	LSTM [44]	17.93	50.05	10.30	41.55
	Transformer [64]	36.65	67.51	15.33	50.39
	GPT-2 [49]	41.73	70.60	27.50	60.36
	CodeGPT [40]	42.18	71.23	28.23	61.81
	CodeGPT-adapted [40]	42.37	71.59	30.60	63.45
	CodeT5-base [66]	36.97	67.12	24.80	58.31
	CodeT5 (Ours) [66]	35.99	66.76	25.20	57.99
	PLBART [1]	38.01	68.46	26.97	61.59
	UniXcoder [15]	43.12	72.00	32.90	65.78
PLM w. RAG	ReACC-bm25 [39]	46.07	73.84	30.63	64.28
	ReACC-dense [39]	45.32	73.95	30.30	64.43
	ReACC-hybrid [39]	46.26	74.41	30.70	64.73
	CONAN	40.12	69.44	26.02	62.86
LLMs	DSCoder-6.7b-Ins	22.65	54.89	17.52	50.39
	DSCoder-6.7b-Ins + CONAN-R	36.67	61.90	24.59	50.40
	DSCoder-6.7b-Ins + CONAN	44.69	73.26	29.80	65.22
	CQwen1.5-7B-Chat	19.40	47.77	15.55	42.60
	CQwen1.5-7B-Chat + CONAN-R	29.80	66.57	24.80	58.31
	CQwen1.5-7B-Chat + CONAN	45.50	73.51	29.00	64.33

The baseline results of PLMs' setting are reported from ReACC [39].

code semantics and fully use the external code knowledge to generate longer codes of the higher-quality, which illustrates the advantages of CONAN in dealing with real-world code generation problems and the possibility of building a code assistant. Besides, utilizing the denoising knowledge of CONAN can further improve the accuracy of code LLMs on code generation tasks.

Additionally, we evaluate the code completion capability of CONAN on PY150 and GitHub JavaCorpus, and the experimental results are shown in Table 7. In our experiment, CONAN does not outperform the state-of-the-art model ReACC (decoder-only architecture), which mainly lies in the different backbone generation models that they use. However, compared to our main baseline model CodeT5, CONAN achieves more than 3% improvements on average, which demonstrates that the supplementary retrieval information is helpful. These T5-based models are pretrained with a span denoising training objective and may be not more tailored for code completion tasks than the auto-regressive generation models, which is also observed in previous work [39, 65]. Furthermore, we can observe that using CONAN as an assistant for code LLMs can achieve competitive results compared to ReACC, which validates the effectiveness of utilizing CONAN to summarize and denoise the retrieved knowledge.

Finally, we further evaluate the performance of CONAN as a code LLMs assistant on HumanEval and MBPP datasets. As shown in Table 8, we observe that using CONAN's denoised knowledge can achieve comparable results to using the top-5 documents retrieved. Moreover, compared to the top-ranked documents, CONAN's denoised results enhance the generation quality of LLMs. This indicates that CONAN possesses the ability to effectively extract relevant information from massive data and denoise them, enabling it to assist LLMs with shorter yet higher-quality texts.

5.2 Ablation Study

In this subsection, we conduct ablation studies to explore the roles of individual modules of CONAN.

Table 8. Code Generation Results on the HumanEval and MBPP Datasets

Dataset	DSCoder	w/ Top-1	w/ Top-5	w/ CONAN	CQwen	w/ Top-1	w/ Top-5	w/ CONAN
HumanEval	<u>75.0</u>	77.4	77.4	77.4	77.9	78.8	80.5	<u>79.3</u>
MBPP	68.9	68.9	70.2	<u>69.9</u>	71.9	70.2	<u>71.2</u>	70.9

DSCoder and CQwen represent Deepseek-Coder-6.7b-Instruct and CodeQwen1.5-7B-Chat respectively. Top- k stands for selecting the top-ranked k code snippets/documentation from CONAN-R retrieval results as external knowledge to assist in code LLMs. The highest results are in **bold** and the second highest scores are underlined.

Table 9. Ablation Study

Methods	Code Generation						Code Summarization		Code Completion			
	CgCSN-P		CgCSN-J		Concode		CsCSN-P	CsCSN-J	PY150		JavaCorpus	
	BLEU	CBLEU	BLEU	CBLEU	BLEU	CBLEU	BLEU	BLEU	EM	ES	EM	ES
CONAN	32.9	37.3	37.7	45.4	42.8	45.1	23.5	26.5	40.1	69.4	26.0	62.9
w/o RAG	6.3	14.8	12.2	17.8	39.6	43.8	20.4	20.5	36.0	66.8	25.2	58.0
w/o FID	27.5	31.7	32.0	36.5	41.9	44.7	22.1	23.8	39.9	67.4	25.4	60.2
w/o Dual-View	30.3	35.1	35.0	41.9	42.9	44.9	23.4	25.9	38.7	68.4	26.5	60.9
CodeBERT + CONAN-G	25.2	31.4	29.0	32.6	41.6	43.4	21.5	21.9	36.7	67.1	25.2	59.6
CodeT5 + CONAN-G	27.5	33.8	33.6	40.3	42.1	44.7	21.9	23.5	38.0	67.5	25.7	61.5

We show the effectiveness of the RAG module, the FID-based dual-view code representation module, and the NL and PL-based code representation method (Dual-View).

As shown in Table 9, we study the effectiveness of generation models using different retrieval-augmented methods, including CONAN w/o RAG, CONAN w/o FID, and CONAN w/o Dual-View. The CONAN w/o RAG model does not incorporate additional code documents during generation, which is the same as the CodeT5. Then the CONAN w/o FID model keeps the same model architecture with CodeT5 and directly concatenates the top-ranked code documents with queries to augment the code/summarization generation capability. CONAN w/o Dual-View only uses the code snippets to augment the model.

Our experimental results show that the advantages of CONAN mainly derive from the external retrieved code knowledge. Compared with CONAN w/o RAG, CONAN w/o FID achieves about 7.6% improvements, showing that the external knowledge benefits the code/summarization generation capability of CONAN. Then the FID model further brings 3% improvements than CONAN w/o FID, which demonstrates the effectiveness of FID architecture in modeling external retrieved knowledge. The improvements mainly lie in that the FID architecture has the ability to overcome the max length limitation of PLMs, denoise the retrieval results, and fully model the external knowledge, which are also observed in previous work [22].

Then we explore the effectiveness of the dual-view code representation method in CONAN-G. Our dual-view-based code document representation method achieves on average 1.85%, 0.05%, and 0.98% improvements on the code generation, code summarization, and code completion tasks, respectively. The better code generation/completion results demonstrate that the code documentation descriptions indeed help the model better understand the code semantics, making the code generation model better copy and refer to the retrieved code snippets to generate more accurate code results. Our dual-view code representation mechanism shows less effectiveness in the code summarization task. The main reason mainly lies in that only 25% of the code snippets have corresponding code documentation in the retrieval database. Additionally, we replace CONAN-R with CodeBERT and CodeT5, which have inferior retrieval performance, and observe a decrease in model performance. This demonstrates that CONAN-R can retrieve higher-quality auxiliary information to guide generation, validating the effectiveness of CONAN-R.

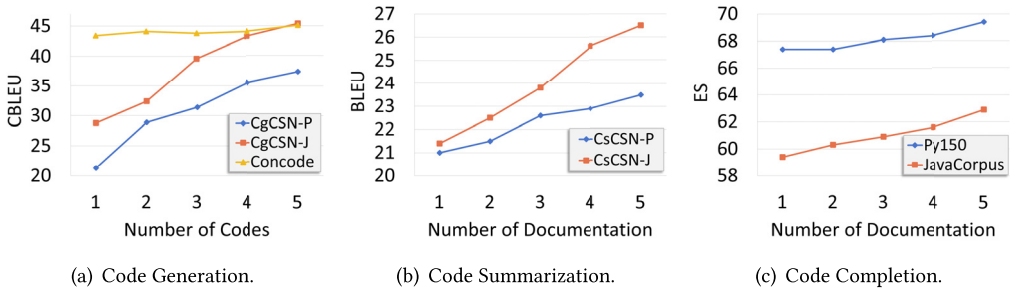


Fig. 4. The impact of the number of retrieved code snippets/documentation on CONAN's performance.

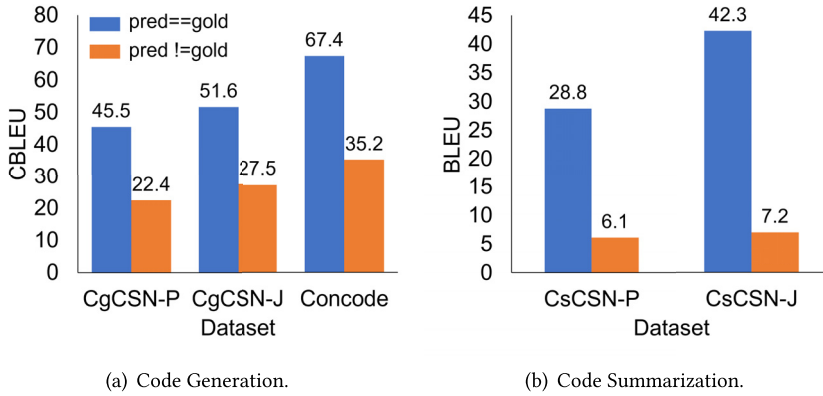


Fig. 5. The similarity between top-1 ranked code documents and the target answers. Based on whether the model's output matches the target answer, the instances in the testing dataset are divided into two groups (pred==gold and pred!=gold). Then the CBLEU score between the top-1 ranked code document and the target answer is calculated for each group. The higher CBLEU/BLEU score indicates the top-1 ranked code document is more similar to the target answer, which illustrates the retrieved code document is of high quality to assist the code generation or summarization tasks.

Finally, we explore the impact of the number of retrieved code snippets/documentation on CONAN's performance. As shown in Figure 4, we observe that increasing the number of retrieved code snippets/documentation leads to a continuous improvement on CONAN's performance in code generation and code summarization. We believe that this is evidence that CONAN excels at combining information from multiple passages.

5.3 The Impact of Retrieved Code Snippets on Code-Related Generation Tasks

In our experiments, we further explore the effectiveness of retrieved code documents in helping CONAN generate code snippets and documentation.

As shown in Figure 5, we group the datasets of the code generation/summarization tasks into two groups according to whether the prediction result is equal to the golden answer. And we denote the two groups as pred==gold and pred!=gold. Then we calculate the average CBLEU or BLEU score between the top-1 ranked retrieved code documents and the target answer to estimate the overlap between the retrieved code documents and the golden answers.

Overall, CONAN achieves double CBLEU/BLEU scores when it correctly predicts the golden answers (pred==gold), showing that more answer-like code documents can provide the necessary knowledge and guide CONAN-G to generate more accurate codes and summarizations. CONAN-G

Table 10. Code Retrieval Performance of CONAN-R

Model	CodeSearch							Adv
	Ruby	Java Script	Go	Python	Java	PHP	Overall	
Zero-Shot								
GraphCodeBERT	1.5	0.4	0.2	0.4	0.7	2.1	0.9	0.5
CodeRetriever	68.7	63.7	87.6	67.7	69.0	62.8	69.1	34.7
CONAN-R	72.6	62.4	88.9	70.0	68.6	62.8	70.9	46.1
Finetuning								
CodeBERT	67.9	62.0	88.2	67.2	67.6	62.8	69.3	27.2
GraphCodeBERT	70.3	64.4	89.7	69.2	69.1	64.9	71.3	35.2
CodeT5	71.9	65.5	88.8	69.8	68.6	64.5	71.5	39.3
CodeRetriever (Inbatch)	75.3	69.5	91.6	73.3	74.0	68.2	75.3	43.0
CodeRetriever (Hard Negative)	75.1	69.8	92.3	74.0	74.9	69.1	75.9	45.1
CONAN-R	74.7	68.6	91.8	73.7	73.7	68.6	75.2	47.3

Because of the GPU memory limitation, we set the batch size as 128 during pretraining and finetuning, which is different from previous work [31]. All models are evaluated on the CodeSearch and Adv datasets and we report the MRR score.

can refer to and copy some code segments from these retrieved code documents to facilitate the generation process, which supports the motivation of building code retrieval-augmented models in code-related tasks. Besides, the `pred! =gold` groups usually achieve higher CBLEU/BLEU scores than the `pred==gold` groups. It demonstrates that these retrieved documents in the `pred! =gold` groups do not help the generation model, which illustrates that the quality of retrieved code documents plays a critical role in guaranteeing the effectiveness of retrieval-augment models.

5.4 Retrieval Effectiveness of Code Structure-Aware Pretraining

In this experiment, we further evaluate the effectiveness of our code structure-aware pretraining method in building a dense retrieval, which includes CDA and MEP. We show the retrieval performance on the code retrieval tasks, then conduct ablation studies, and finally visualize the embedding space.

Retrieval Effectiveness in the Code Retrieval Tasks. We show the effectiveness of our code structure-aware pretraining method by evaluating the pretrained models in the code retrieval tasks. In this experiment, we follow previous work [32] and use Adv and CodeSearch datasets for training and evaluation.

As shown in Table 10, we start from the code structure-aware pretrained model and then finetune the model using different datasets. In the zero-shot setting, CONAN-R outperforms CodeRetriever with about 2% improvements and 14% on CodeSearch and Adv, showing the effectiveness of our code structure-aware pretraining method. Notably, CONAN-R also shows strong zero-shot ability by achieving comparable performance with the finetuned CodeBERT, GraphCodeBERT, and CodeT5 models. After finetuning, CONAN-R achieves 3.7% and 9.3% improvements over CodeT5 on CodeSearch and Adv, respectively. The improvements demonstrate that our pretraining strategy has the ability to enable PLMs to better represent code data and bring its advantages to the downstream code-related retrieval tasks.

Effectiveness of Different Pretraining Strategies. Then we explore the effectiveness of pretraining strategies in teaching the CodeT5 model to represent the code for retrieval.

As shown in Table 11, We start from CodeT5 models and continuously train CodeT5 using two proposed training tasks, MEP and CDA to show their effectiveness. Meanwhile, we compare the

Table 11. The Retrieval Performance of Ablation Models of Our Code Structure-Aware Pretraining Method on Adv Dataset

Model	CodeT5			CONAN-R	
	Vanilla	w/ MEP	w/ CDA	Span Mask	Entity Mask
Zero-Shot	0.03	0.03	45.01	35.88	46.08
Finetuning	39.30	38.46	46.98	42.11	47.28

MEP and CDA are two tasks for pretraining CONAN-R, which are proposed by our previous work [32].

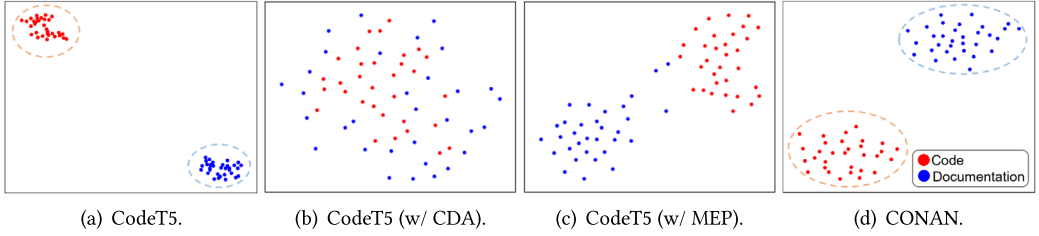


Fig. 6. Embedding visualization of different models using T-SNE. We randomly sample 32 code snippets (Code) and 32 code documentation (Documentation) from the testing set of the Adv dataset and plot their embedding distribution.

MEP method with the random span masking strategy [50, 66] to evaluate the effectiveness of different mask modeling strategies. The retrieval performance in both zero-shot and finetuning settings is shown.

Compared with the vanilla CodeT5 model, MEP and CDA show distinct performance in code retrieval. As expected, MEP shows almost the same performance as the baseline model. It shows that only mask language modeling usually shows less effectiveness in learning representations for code data, even using different masking strategies. Different from MEP, CDA shows significant improvements in the code retrieval task. Our CDA training method contrastively trains CodeT5 models using the alignment relations between code and NL, which helps to bridge the modality gap between them, maps code and NL in one universal embedding space, and learns more effective representations for retrieval. When adding additional task MEP to CodeT5 (w/ CDA), the retrieval performance of CONAN-R is consistently improved. This phenomenon shows that mask language modeling is still effective in teaching CodeT5 to better capture the structure semantics in code and conduct more effective text representations for code by filling up the masked entities.

We also compare different masking strategies that are used during mask language modeling. Our entity masking strategy outperforms the random span masking strategy, showing the crucial role of entities in code data understanding. Besides, CONAN-R pretrained using the MEP task achieves comparable ranking performance with finetuned models, which illustrates that structure-aware pretraining can directly benefit downstream tasks, such as code retrieval.

Embedding Visualization. Finally, we present the embedding distribution of documentation texts and their corresponding codes in Figure 6.

Overall, depending on our code structure-aware pretraining methods, CONAN conducts a more uniform embedding space than CodeT5 and makes the representations of code snippets and documentation more distinguished in the embedding space. Then we analyze the effectiveness of our continuous training methods, MEP, and CDA. By comparing Figure 6(b) with Figure 6(a), our CDA task indeed helps PLMs to align the representations of code snippets and documentation,

Table 12. Case Studies

Code Generation Task
Input: Retrieves connection to Cloud Translate
Top1-Ranked Code Snippet: <code>def _get_connection (self) : if not hasattr (self , “_connection”) : if self . use_pyrax : public = not self . use_snet # invert self . _connection = pyrax . connect_to_cloudfiles (public = public) elif swiftclient : self . _connection = swiftclient . Connection (authurl = self . auth_url , user = self . username , key = self . api_key , snet = self . use_snet , auth_version = self . auth_version , tenant_name = self . auth_tenant_name ,) else : raise NotImplementedError (“Cloud connection is not correctly configured.”) return self . _connection</code>
Top2-Ranked Code Snippet: <code>def cloudwatch_connection (self) : if self . _cloudwatch_connection is None : conn = self . _session . create_client (“cloudwatch,” self . connection . region) self . _cloudwatch_connection = conn return self . _cloudwatch_connection</code>
CodeT5: <code>def get_translator (self , max_segments = 128 , from_lang = None) : if not googleapiclient : raise ImportError (“You must install google-apiclient to use translate.”) if not self . developer_key : raise Exception (“You must install google-apiclient to use translate.”) if not self . service : raise Exception (“You must install google-apiclient to use translate.”) url_base = “https://api.googleapis.com/language/translate/v2” params = “apikey” : self . developer_key if from_lang : params [‘from’] = from_lang response = requests . get (url_base , params = params) self . translated_strings = response . json () return self . translated_strings</code>
CONAN: <code>def get_cloud_translate_connection (self) : if not self . _cloud_translate_connection : self . _cloud_translate_connection = CloudTranslateConnectionClient (credentials = self . _get_credentials ()) return self . _cloud_translate_connection</code>
Ground Truth: <code>def get_conn (self) : if not self . _client : self . _client = Client (credentials = self . _get_credentials ()) return self . _client</code>
Code Summarization Task
Input: <code>def create_instance (self , body , project_id = None) : response = self . get_conn () . instances () . insert (project = project_id , body = body) . execute (num_retries = self . num_retries) operation_name = response [“nam”] self . _wait_for_operation_to_complete (project_id = project_id , operation_name = operation_name)</code>
Top1-Ranked Document: Create an instance within a project.
Top2-Ranked Document: InsertInstance creates a new instance on GCP.
CodeT5: Create an instance within a project.
CONAN: Create a new SQL instance.
Ground Truth: Creates a new Cloud SQL instance.
Code Completion Task
Input: <code>import unittest from contextlib import contextmanager import logging import os from path import path import shovel import sys class TestRun(unittest.TestCase): “<STR_LIT>” def logs(self, pth, *args, **kwargs): with path(pth): with logs() as out: shovel.run(*args, **kwargs) return [line.strip() for line in out.getvalue().strip().split(“<STR_LIT:>”)] def test_verbose(self): “<STR_LIT>” actual = self.logs(“<STR_LIT>”, “<STR_LIT:bar>”, “<STR_LIT>”) actual</code>
Top1-Ranked Document: Replace the current path with the given unencoded path.
Top2-Ranked Document: Replace absolute urls with relative path.
CodeT5: = [
CONAN: = [line.strip(). replace (os.getcwd(), “<STR_LIT>”) for line in actual]
Ground Truth: = [line. replace (os.getcwd(), “<STR_LIT>”) for line in actual]

We randomly sample three cases from code generation, summarization, and completion tasks to show the effectiveness of CONAN. The matched parts are **emphasized**. The predictions of CodeT5 are generated directly based on the input, without utilizing any retrieved information.

which reduces the distance between matched code-documentation pairs and mixes the multi-modal embeddings thoroughly in the embedding space. After adding the MEP training task to CodeT5 (w/ CDA) (from Figure 6(b)–(d)), the embedding distributions of code snippets and documentation become distinguished again, demonstrating that MEP can help models capture different semantics from different data modalities to represent them. Besides, by comparing Figure 6(d) with Figure 6(c), the CDA task also makes the boundary of the embedding clusters of code snippets and documentation clearer. The main reason lies in that these embeddings are assigned to appropriate positions for aligning matched code-documentation pairs with the help of our CDA task.

5.5 Case Study

Finally, we show three cases from code generation, summarization, and completion tasks to show the effectiveness of CONAN in Table 12.

For the first case, CONAN retrieves some related code snippets that include some related API/function usages, such as “connect_to_cloudfiles” and “create_client,” which aims to retrieve a connection to Cloud Translate. These API/function usage examples help the generation CONAN directly implement the “get_cloud_translate_connection” function instead of generating some redundant judgment statements. The second case is an example of the code summarization task. In this case, CodeT5 generates a more general summarization “Create an instance within a project.” CONAN retrieves a general documentation description and a more detailed one, which helps CONAN better understand the summarizations and codes and then generate a specific function summary “create a SQL instance.” For the code completion case, CodeT5 only generates “[,” showing that the CodeT5 model is not skilled in the code completion task and CONAN demonstrates its utility in completing the unfinished code. CONAN retrieves some related code documents that replace the path or URL and then correctly generates the golden answer, showing that the related code documents indeed help CONAN complete the unfinished codes.

6 Conclusion

This article proposes CONAN, which aims to help human and LLMs to solve different code-related tasks. CONAN constructs a retrieval-augmented architecture that generalizes to multiple code generation tasks by designing a code structure-aware retriever (CONAN-G) and a dual-view code representation method for building a generation model (CONAN-G). Our experiments show the code document retrieval augmented method is effective in improving the code/documentation generation ability of CodeT5. The improvements derive from a more effective code retriever (CONAN-R) and a better code understanding of the generation model (CONAN-G) by using the code documentation as the code gist. The experimental results on different code-related tasks show the potential advantages of CONAN in building a real-world code assistant by employing the RAG framework.

References

- [1] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of NAACL-HLT*, 2655–2668.
- [2] Miltiadis Allamanis and Charles Sutton. 2013. Mining source code repositories at massive scale using language modeling. In *Proceedings of MSR*, 207–216.
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models.
- [4] Brenda S. Baker. 2007. Finding clones with dup: Analysis of an experiment. *IEEE Transactions on Software Engineering* 9 (2007), 608–621.
- [5] Joel Brandt, Mira Dontcheva, Marcos Weskamp, and Scott R. Klemmer. 2010. Example-centric programming: Integrating web search into the development environment. In *Proceedings of CHI*, 513–522.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf,

- Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code.
- [7] Xinlei Chen and Kaiming He. 2021. Exploring simple Siamese representation learning. In *Proceedings of CVPR*, 15750–15758.
 - [8] Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. 2020. ELECTRA: Pre-training text encoders as discriminators rather than generators. In *Proceedings of ICLR*.
 - [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 4171–4186.
 - [10] Hongchao Fang, Sicheng Wang, Meng Zhou, Jiayuan Ding, and Pengtao Xie. 2020. Cert: Contrastive self-supervised learning for language understanding.
 - [11] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Proceedings of EMNLP Findings*, 1536–1547.
 - [12] Jun Gao, Di He, Xu Tan, Tao Qin, Liwei Wang, and Tie-Yan Liu. 2019. Representation degeneration problem in training natural language generation models. In *Proceedings of ICLR*.
 - [13] Luyu Gao and Jamie Callan. 2021. Condenser: A pre-training architecture for dense retrieval. In *Proceedings of EMNLP*, 981–993.
 - [14] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of EMNLP*, 6894–6910.
 - [15] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of ACL*, 7212–7225.
 - [16] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *Proceedings of ICLR*.
 - [17] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the large language model meets programming – The rise of code intelligence.
 - [18] Yucan Guo, Zixuan Li, Xiaolong Jin, Yantao Liu, Yutao Zeng, Wenxuan Liu, Xiang Li, Pan Yang, Long Bai, Jiafeng Guo, and Xueqi Cheng. 2023. Retrieval-augmented code generation for universal information extraction.
 - [19] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Retrieval augmented language model pre-training. In *Proceedings of ICML*, 3929–3938.
 - [20] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2020. CodeSearchNet challenge: Evaluating the state of semantic code search.
 - [21] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping language to code in programmatic context. In *Proceedings of EMNLP*, 1643–1652.
 - [22] Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of EACL*, 874–880.
 - [23] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation.
 - [24] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 21 (2019), 535–547.
 - [25] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 6769–6781.
 - [26] Marie-Anne Lachaux, Baptiste Rozière, Marc Szafraniec, and Guillaume Lample. 2021. DOBF: A deobfuscation pre-training objective for programming languages. In *Proceedings of NeurIPS*, 14967–14979.
 - [27] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of NeurIPS*.
 - [28] Bohan Li, Hao Zhou, Junxian He, Mingxuan Wang, Yiming Yang, and Lei Li. 2020. On the sentence embeddings from pre-trained language models. In *Proceedings of EMNLP*, 9119–9130.
 - [29] Hongwei Li, Zhenchang Xing, Xin Peng, and Wenyun Zhao. 2013. What help do developers seek, when and how? In *Proceedings of WCRE*, 142–151.

- [30] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. SkCoder: A sketch-based approach for automatic code generation.
- [31] Xiaonan Li, Yeyun Gong, Yelong Shen, Xipeng Qiu, Hang Zhang, Bolun Yao, Weizhen Qi, Daxin Jiang, Weizhu Chen, and Nan Duan. 2022. CodeRetriever: A large scale contrastive pre-training method for code search. In *Proceedings of EMNLP*, 2898–2910.
- [32] Xinze Li, Zhenghao Liu, Chenyan Xiong, Shi Yu, Yu Gu, Zhiyuan Liu, and Ge Yu. 2023. Structure-aware language model pretraining improves dense retrieval on structured data. In *Proceedings of ACL*, 11560–11574.
- [33] Yizhi Li, Zhenghao Liu, Chenyan Xiong, and Zhiyuan Liu. 2021. More robust dense retrieval with contrastive dual learning. In *Proceedings of SIGIR*, 287–296.
- [34] Dianshu Liao, Shidong Pan, Qing Huang, Xiaoxue Ren, Zhenchang Xing, Huan Jin, and Qinying Li. 2023. Context-aware code generation framework for code repositories: Local, global, and third-party library awareness.
- [35] Chin-Yew Lin and Franz Josef Och. 2004. ORANGE: A method for evaluating automatic evaluation metrics for machine translation. In *Proceedings of COLING*, 501–507.
- [36] Shangqing Liu, Yu Chen, Xiaofei Xie, Jing Kai Siow, and Yang Liu. 2021. Retrieval-augmented generation for code summarization via hybrid GNN. In *Proceedings of ICLR*.
- [37] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized BERT pretraining approach.
- [38] Zhenghao Liu, Chenyan Xiong, Yuanhuiyi Lv, Zhiyuan Liu, and Ge Yu. 2023. Universal vision-language dense retrieval: Learning a unified representation space for multi-modal retrieval. In *Proceedings of ICLR*.
- [39] Shuai Lu, Nan Duan, Hojae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. 2022. ReACC: A retrieval-augmented code completion framework. In *Proceedings of ACL*, 6227–6240.
- [40] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of NeurIPS*.
- [41] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. 2021. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics* (2021), 329–345.
- [42] Hongyin Luo, Yung-Sung Chuang, Yuan Gong, Tianhua Zhang, Yoon Kim, Xixin Wu, Danny Fox, Helen Meng, and James Glass. 2023. SAIL: Search-augmented instruction learning.
- [43] Thang Luong, Hieu Pham, and Christopher D. Manning. 2015. Effective approaches to attention-based neural machine translation. In *Proceedings of EMNLP*, 1412–1421.
- [44] Sepp Hochreiter and Jürgen Schmidhuber. 2010. Long short-term memory. *Neural Computation* 8 (2010), 1735–1780.
- [45] Yu Meng, Chenyan Xiong, Payal Bajaj, Saurabh Tiwary, Paul Bennett, Jiawei Han, and Xia Song. 2021. COCO-LM: Correcting and contrasting text sequences for language model pretraining. In *Proceedings of NeurIPS*, 23102–23114.
- [46] OpenAI. 2022. ChatGPT: Optimizing Language Models for Dialogue. Retrieved from <https://proceedings.neurips.cc/paper/2021/hash/c2c2a04512b35d13102459f8784f1a2d-Abstract.html>
- [47] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of ACL*, 311–318.
- [48] Md Rizwan Parvez, Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. In *Proceedings of EMNLP Findings*, 2719–2734.
- [49] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 8 (2019), 9.
- [50] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21 (2020), 140:1–140:67.
- [51] Veselin Raychev, Pavol Bielik, and Martin Vechev. 2016. Probabilistic model for code with decision trees. *ACM SIGPLAN Notices* (2016), 731–747.
- [52] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP*, 3982–3992.
- [53] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: A method for automatic evaluation of code synthesis.
- [54] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 4 (2009), 333–389.
- [55] Chanchal K. Roy and James R. Cordy. 2008. An empirical study of function clones in open source software. In *Proceedings of WCRE*, 81–90.
- [56] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian

- Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open foundation models for code.
- [57] Caitlin Sadowski, Kathryn T. Stolee, and Sebastian Elbaum. 2015. How developers search for code: A case study. In *Proceedings of FSE*, 191–201.
- [58] Christopher Sciavolino, Zexuan Zhong, Jinhyuk Lee, and Danqi Chen. 2021. Simple entity-centric questions challenge dense retrievers. In *Proceedings of EMNLP*, 6138–6148.
- [59] Anton Shapkin, Denis Litvinov, and Timofey Bryksin. 2023. Entity-augmented code generation.
- [60] Disha Shrivastava, Denis Kocetkov, Harm de Vries, Dzmitry Bahdanau, and Torsten Scholak. 2023. RepoFusion: Training code models to understand your repository.
- [61] Jeffrey Svajlenko and Chanchal K. Roy. 2015. Evaluating clone detection tools with BigCloneBench. In *Proceedings of ICSME*, 131–140.
- [62] Qwen Team. 2024. Code with CodeQwen1.5. DOI: <https://doi.org/10.1109/ICSM.2015.7332459>
- [63] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutvi Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models.
- [64] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of NeurIPS*, 5998–6008.
- [65] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation.
- [66] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of EMNLP*, 8696–8708.
- [67] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace’s transformers: State-of-the-art natural language processing.
- [68] Zhuofeng Wu, Sinong Wang, Jiatao Gu, Madian Khabsa, Fei Sun, and Hao Ma. 2020. Clear: Contrastive learning for sentence representation.
- [69] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *Proceedings of ICLR*.
- [70] Wenhan Xiong, Xiang Lorraine Li, Srinu Iyer, Jingfei Du, Patrick S. H. Lewis, William Yang Wang, Yashar Mehdad, Scott Yih, Sebastian Riedel, Douwe Kiela, and Barlas Oguz. 2021. Answering complex open-domain questions with multi-hop dense retrieval. In *Proceedings of ICLR*.
- [71] Yuanmeng Yan, Rumei Li, Sirui Wang, Fuzheng Zhang, Wei Wu, and Weiran Xu. 2021. ConSERT: A contrastive framework for self-supervised sentence representation transfer. In *Proceedings of ACL*, 5065–5075.
- [72] Deming Ye, Yankai Lin, Jiaju Du, Zhenghao Liu, Peng Li, Maosong Sun, and Zhiyuan Liu. 2020. Coreferential reasoning learning for language representation. In *Proceedings of EMNLP*, 7170–7186.
- [73] Shi Yu, Zhenghao Liu, Chenyan Xiong, Tao Feng, and Zhiyuan Liu. 2021. Few-shot conversational dense retrieval. In *Proceedings of SIGIR*.
- [74] Shi Yu, Zhenghao Liu, Chenyan Xiong, and Zhiyuan Liu. 2023. OpenMatch-v2: An all-in-one multi-modality PLM-based information retrieval toolkit. In *Proceedings of SIGIR*, 3160–3164.
- [75] Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. CERT: Continual pre-training on sketches for library-oriented code generation. In *Proceedings of IJCAI*, 3160–3164.
- [76] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-level code completion through iterative retrieval and generation.
- [77] Zhengyan Zhang, Xu Han, Zhiyuan Liu, Xin Jiang, Maosong Sun, and Qun Liu. 2019. ERNIE: Enhanced language representation with informative entities. In *Proceedings of ACL*, 1441–1451.
- [78] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhengbao Jiang, and Graham Neubig. 2022. DocPrompting: Generating code by retrieving the docs. In *Proceedings of ICLR*, 1441–1451.

Received 8 January 2024; revised 25 July 2024; accepted 31 August 2024